

A class attribute is a variable that belongs to the class rather than to an object. We create class attributes inside the class but outside all methods, just like we create global variables in a `.py` file but outside all functions. Here's an example of a class attribute named `count`, which keeps track of how many `CreateCounter` objects have been created:

```
class CreateCounter:
    count = 0 # This is a class attribute.

    def __init__(self):
        CreateCounter.count += 1

print('Objects created:', CreateCounter.count) # Prints 0.
a = CreateCounter()
b = CreateCounter()
c = CreateCounter()
print('Objects created:', CreateCounter.count) # Prints 3.
```

The `CreateCounter` class has a single class attribute named `count`. All `CreateCounter` objects share this attribute rather than having their own separate `count` attributes. This is why the `CreateCounter.count += 1` line in the constructor function can keep count of every `CreateCounter` object created. When you run this program, the output will look like this:

```
Objects created: 0
Objects created: 3
```

We rarely use class attributes. Even this “count how many `CreateCounter` objects have been created” example can be done more simply by using a global variable instead of a class attribute.

Static Methods

A *static method* doesn't have a `self` or `cls` parameter. Static methods are effectively just functions, because they can't access the attributes or methods of the class or its objects. Rarely, if ever, do you need to use static methods in Python. If you do decide to use one, you should strongly consider just creating a regular function instead.